

Hafta 3 — Derinlemesine

⚠ BU DOSYA SINAV KAPSAMI DIŐINDADIR

Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, "peki bu nasıl çalışıyor?" diye soran öğrenci içindir. Sınav kapsamı **H03-ders-notu.md** dosyasıdır.

Dönem projesinde işinize yarayacak birkaç şey de buradadır (§3, §6).

1. **.cshtml** Hangi C# Sınıfına Dönüşür?

Ders notu §1.1'de "Razor dosyası önceden derlenmiş bir C# sınıfıdır" dedik. Bu, mecaz değil; gerçekten bir sınıf üretilir.

Őu görünüm:

```
@model List<KampusKitap.Models.Kitap>
<h1>Kitaplar</h1>
@foreach (var k in Model)
{
    <li>@k.Baslik</li>
}
```

kabaca őuna dönüşür:

```
public class Views_Kitap_Index : RazorPage<List<KampusKitap.Models.Kitap>>
{
    public override async Task ExecuteAsync()
    {
        WriteLiteral("<h1>Kitaplar</h1>\n");
        foreach (var k in Model)
        {
            WriteLiteral("<li>");
            Write(k.Baslik);           // ← HTML KODLAYAN yazım
            WriteLiteral("</li>\n");
        }
    }
}
```

Buradan üç şey okunur:

Gözlem	Sonucu
--------	--------

<code>WriteLiteral</code> ile <code>Write</code> farklı metotlar	<code>WriteLiteral</code> ham HTML basar (sizin yazdığınız etiketler); <code>Write</code> kodlar (veri)
<code>@Html.Raw(x)</code> → <code>Write(x)</code> yerine <code>WriteLiteral</code> - benzeri davranış	Kodlama bu yüzden atlanır (ders notu §1.5)
Sınıf <code>RazorPage<TModel></code> türetir	<code>Model</code> , <code>ViewData</code> , <code>Html</code> , <code>Url</code> bu taban sınıftan gelir

`@{ }` blokları metot gövdesine olduğu gibi yazılır; `@if` / `@foreach` ise gövdeyi `WriteLiteral` çağrısıyla bölerek C# denetim yapısına dönüştür. Razor'un "`@`"tan sonrası C#" kuralı, aslında **parser'ın HTML modu ile C# modu arasında geçiş yapmasıdır**.

2. Derleme Zamanı mı, Çalışma Zamanı mı?

ASP.NET Core'da `.cshtml` dosyaları **derleme (build) ve yayın (publish) zamanında** Razor SDK tarafından derlenir. Yani `dotnet publish` çıktısında görünümünüz artık `.cshtml` metni değil, derlenmiş koddur.

Çalışma zamanı derleme (runtime compilation) ayrı bir pakettir

(`Microsoft.AspNetCore.Mvc.Razor.RuntimeCompilation`) ve dosyayı siz çalışırken değiştirdiğinizde yeniden derler.

⚠ .NET 10'a özel: runtime compilation artık obsolete

Microsoft, Razor **çalışma zamanı derlemesini .NET 10 ile birlikte kullanım dışı (obsolete)** ilan etti. Gerekçeleri:

- **.NET Hot Reload**'u devre dışı bırakır
- `global using` ve örtük (implicit) `using` yönergelerini desteklemez
- Blazor bileşenlerinde desteklenmez
- Üretim ortamı için önerilmez

Bugünün doğru yolu: geliştirmede **Hot Reload**, üretimde **derleme zamanı derleme**. İnternette bulacağınız "görünümleri canlı düzenlemek için AddRazorRuntimeCompilation ekleyin" tavsiyesi .NET 10 için **eskimiştir**.

Pratik sonuç: derste `.cshtml` değişikliğini görmek için çoğu zaman `dotnet watch` yeterlidir; görmüyorsanız uygulamayı yeniden başlatın.

Kaynak: [Razor file compilation in ASP.NET Core](#)

3. Görünüm Arama Sırasını Değiştirmek

Ders notu §2.5'te iki yol gördük. Bu sıra sabit değildir; `IViewLocationExpander` ile genişletilebilir. Tipik kullanım: çok kiracılı (multi-tenant) bir uygulamada her müşteriye kendi görünümünü vermek.

```
public class TemaGenisletici : IViewLocationExpander
{
    public void PopulateValues(ViewLocationExpanderContext context)
    {
        context.Values["tema"] = context.ActionContext.HttpContext.Request.Host.Host;
    }

    public IEnumerable<string> ExpandViewLocations(
        ViewLocationExpanderContext context, IEnumerable<string> viewLocations)
    {
        var tema = context.Values["tema"];
        // Önce temaya özel yol, sonra varsayılanlar
        return new[] { $"/Temalar/{tema}/{1}/{0}.cshtml" }.Concat(viewLocations);
    }
}
```

```
builder.Services.Configure<RazorViewEngineOptions>(o =>
    o.ViewLocationExpanders.Add(new TemaGenisletici()));
```

Yer tutucular: `{0}` görünüm adı, `{1}` controller adı, `{2}` alan (area) adı.

💡 Ders notundaki hata mesajının listelediği yollar, bu genişleticilerden **sonra** oluşan nihai listedir. Yani bir genişletici eklediyseniz, hata mesajı onu da gösterir.

4. ViewComponent 'in Döndürebileceği Sonuçlar

Ders notu §3.2'de yalnızca `View(...)` kullandık. Taban sınıfın üç yardımcısı vardır:

Dönüş	Ne yapar	Ne zaman
<code>View()</code> / <code>View(model)</code>	<code>Default.cshtml</code> 'i render eder	Olağan durum
<code>View("Kompakt", model)</code>	Aynı klasörde <code>Kompakt.cshtml</code>	Aynı bileşenin iki görünümü
<code>Content("metin")</code>	Kodlanmış düz metin	Tek satırlık çıktı; görünüm dosyasına değmez

```
public IViewComponentResult Invoke(bool kompakt)
=> View(kompakt ? "Kompakt" : "Default", _sayac.Oku());
```

Invoke mi InvokeAsync mi? Çatı ikisini de bulur. Kural basittir: içeride `await` edeceğiniz bir iş varsa (veritabanı, HTTP) `InvokeAsync`, yoksa `Invoke`. Asenkron olmayan bir işi `async` yapmak hiçbir şey kazandırmaz.

View Component bir controller değildir:

- Route'lanmaz, adresi yoktur, doğrudan HTTP isteği alamaz

- Model binding ve filtre (filter) hattından geçmez
- `Request` 'e erişebilir, ama ona karar verdimen — parametre alması daha iyidir

5. `PartialAsync` ve `RenderPartialAsync` Farkı

Üç yazım da aynı partial'ı çağırır:

```
<partial name="_KitapKart" model="k" />           @* Tag Helper – tercih edilen *@
@await Html.PartialAsync("_KitapKart", k)         @* sonucu DÖNDÜRÜR *@
@{ await Html.RenderPartialAsync("_KitapKart", k); } @* doğrudan çıktıya YAZAR *@
```

	<code>PartialAsync</code>	<code>RenderPartialAsync</code>
Sonuç	<code>IHtmlContent</code> döndürür	Doğrudan yanıt akışına yazar
Yazım	<code>@await ...</code>	<code>@{ await ...; }</code> — süslü parantez zorunlu
Bellek	Ara tampon oluşur	Oluşmaz, biraz daha hızlı
Değişkene atanabilir mi	Evet	Hayır

Fark ölçülebilir olmak için binlerce yineleme gerekir. **Tag Helper yazımını kullanın**; diğer ikisi eski kodda karşınıza çıkacağı için burada.

⚠ `@Html.Partial(...)` (senkron, `await` siz) sürümü **kullanılmamalıdır** — asenkron bağlamda kilitlenmeye (deadlock) yol açabildiği için analiz uyarısı verir.

6. `_ViewStart` Zinciri ve Çalışma Sırası

Birden fazla `_ViewStart.cshtml` varsa hepsi çalışır, **kökten yaprağa doğru**:

```
Views/_ViewStart.cshtml      ← önce
Views/Kitap/_ViewStart.cshtml ← sonra (öncekini EZEBİLİR)
Views/Kitap/Index.cshtml     ← en son
```

Yani `Views/Kitap/_ViewStart.cshtml` içinde `Layout = "_KitapLayout"`; yazarsanız, yalnızca o klasördeki görünümeler farklı layout kullanır. Kökteki dosya yine çalışır — ama değeri ezilir.

`_ViewImports.cshtml` için de aynı zincir geçerlidir, bir farkla: yönergeler **birikir** (`@using` 'ler toplanır), ezilmez. İstisna `@model` ve `@inherits`: onlarda en yakın olan kazanır.

Katmanlı bir layout da kurulabilir — layout'un kendi layout'u olabilir:

```
@* Views/Shared/_AdminLayout.cshtml *@
@{
    Layout = "_Layout";          @* site layout'unun içine yerleşir *@
}
<div class="admin-kenar">...</div>
@RenderBody()
```

7. ViewData / ViewBag ile Layout'a Veri Taşımak

Layout, görünümün modelini bilmez (bilmemelidir de). Ortak alan **ViewData** 'dır:

```
@* Görünümde *@
@{
    ViewData["Title"] = "Kitaplar";
}
```

```
@* Layout'ta *@
<title>@ViewData["Title"] - KampusKitap</title>
```

ViewBag, **ViewData** 'nın **dynamic** kabuğudur: **ViewBag.Title** ile **ViewData["Title"]** **aynı sözlüğe** yazar. **dynamic** olduğu için yazım hatası derlemede yakalanmaz.



ViewData, **ViewBag** ve **TempData** üçlüsü **Hafta 7**'de (state management ile birlikte) ayrıntılı işlenir. Burada yalnızca layout'a başlık taşıdığımız için var.

Daha sağlam yol, layout'un kendi modelini bir View Component ile beslemektir — bu hafta öğrendiğiniz nedenle: **veriyi kendi getirir**.

8. HTML Kodlaması: Bağlam Önemlidir

Ders notu \$1.5'te Razor'ın otomatik kodlamasını gördük. Eksik kalan nokta şu: **kodlama bağlama göre değişir**.

Veri nereye giriyor	Doğru kodlama	Razor otomatik yapar mı
HTML gövdesi (<code><p>@x</p></code>)	HTML encoding	✓ evet
HTML özniteliği (<code>title="@x"</code>)	HTML attribute encoding	✓ evet
JavaScript içi (<code><script>var a = "@x";</script></code>)	JavaScript encoding	✗ hayır — tehlikeli
URL sorgu dizesi	URL encoding	✗ hayır

Üçüncü satır gerçek bir tuzaktır. Doğru yol, değeri **bir veri özniteliğine** koyup JavaScript'te okumaktır:

```
<div id="veri" data-baslik="@Model.Baslik"></div>
<script>
  const baslik = document.getElementById("veri").dataset.baslik;
</script>
```

Kod içinde kodlayıcılara DI ile de erişilebilir: `HtmlEncoder`, `JavaScriptEncoder`, `UrlEncoder`.

Kodlama çıktı anında yapılır, veritabanında saklanmaz. Kodlanmış veriyi saklarsanız aramak, karşılaştırmak ve başka bir bağlamda (e-posta, PDF) kullanmak bozulur.

Kaynak: [Prevent XSS in ASP.NET Core](#)

9. Razor Pages ile ilişkisi

Bu derste **MVC** kullanıyoruz: controller + görünüm. ASP.NET Core'un ikinci bir sayfa modeli daha var: **Razor Pages**.

	MVC	Razor Pages
Dosya	<code>Controllers/KitapController.cs</code> + <code>Views/Kitap/Index.cshtml</code>	<code>Pages/Kitap/Index.cshtml</code> + <code>Index.cshtml.cs</code>
Yönlendirme	Route kalıbı	Dosya yolu doğrudan URL
Kod nerede	Controller'da, action'larda	Sayfanın yanındaki <code>PageModel</code> sınıfında

Ortak olan: Razor sözdizimi, Layout, `_ViewStart`, `_ViewImports`, partial, View Component. Yani bu haftanın öğrendiklerinin **tamamı** Razor Pages'te de aynen geçerlidir.

Bu dersin MVC'yi seçme gerekçesi `H02-detay.md` 'dedir: sınav ve EBS izlencesi MVC üzerine kuruludur, ayrıca controller/action ayrımı Hafta 12'deki Web API'ye doğrudan köprü kurar.

10. Daha İleri Gitmek İsteyenlere

Konu	Nerede
Razor sözdizimi — tam referans	https://learn.microsoft.com/aspnet/core/mvc/views/razor?view=aspnetcore-10.0
Görünüm derlemesi ve runtime compilation'ın durumu	https://learn.microsoft.com/aspnet/core/mvc/views/view-compilation?view=aspnetcore-10.0
View component — tüm senaryolar	https://learn.microsoft.com/aspnet/core/mvc/views/view-components?view=aspnetcore-10.0
Partial view — üç çağırma biçimi	https://learn.microsoft.com/aspnet/core/mvc/views/partial?view=aspnetcore-10.0

XSS — kodlayıcıları özelleştirmek	https://learn.microsoft.com/aspnet/core/security/cross-site-scripting?view=aspnetcore-10.0
Razor class library (görünümleri paketlemek)	https://learn.microsoft.com/aspnet/core/razor-pages/ui-class?view=aspnetcore-10.0

⚠️ Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**. Sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür ve `Startup.cs` 'li örneklerle karşılaşabilirsiniz.